

BrokerVerse

Out-of-the-box insurance broking platform

Solution Architecture · Document 06 of 11

Capacity and Performance

Product	BrokerVerse
Document	06 Capacity and Performance
Version	1.0
Date	29 September 2026
Prepared by	iorta TechNXT
Classification	Confidential: for the client, iorta TechNXT and authorised partners

Document control

Document	BrokerVerse Solution Architecture: 06 Capacity and Performance
Document ID	BV-SA-06
Version	1.0
Status	Issued for review
Date	29 September 2026
Prepared by	iorta TechNXT
Source baseline	BrokerVerse source repository, branch brokerverse-platform (backend/, brokerverse/, docker-compose.yml, render.yaml, docs/)

Version history

Version	Date	Author	Change
1.0	29 September 2026	iorta TechNXT	Initial issue

Review and approval

Role	Name	Signature	Date
Solution architect			
DevOps / infrastructure lead			
Information security			
Business owner			

Conventions

- Statements of fact are derived from the code, configuration and database of the baseline above (file paths are given relative to the repository). Figures measured on the local test system are labelled **test data** or **indicative**.
- Targets, sizes, retention periods and thresholds that are not facts of the code are labelled **Recommended / to be confirmed by the business and DevOps**.
- Items marked **Gap** are defects or missing capabilities found while preparing this document.

Contents

1 Introduction.....	4
1.1 Purpose and scope.....	4
2 Built-in capacity controls.....	5
2.1 Request and upload limits.....	5
2.2 Performance mechanisms.....	5
2.3 Known performance risks in the code.....	6
3 Indicative load test.....	7
3.1 Method.....	7
3.2 Results.....	7
3.3 Interpretation.....	8
4 Data growth and storage.....	9
4.1 Current relative weight (test data).....	9
4.2 Growth estimate.....	9
5 Sizing recommendation.....	11
6 Performance test plan for UAT.....	12
6.1 Objectives.....	12
6.2 Response-time targets.....	12
6.3 Approach.....	12
7 Related documents.....	13
8 Glossary.....	14

1 Introduction

1.1 Purpose and scope

This document describes the capacity and performance characteristics of BrokerVerse: the built-in limits and performance mechanisms of the code, the results of an indicative load test on the development container, the expected data growth, a sizing recommendation for the AWS target, and the performance test plan for UAT.

Recommended / to be confirmed by the business and DevOps: The business volumes used for sizing in this document (users, policies per year, documents per policy) are assumptions. Replace them with the business forecast before the infrastructure is ordered, and confirm the sizing with the UAT performance test in chapter 6.

Built-in capacity controls

2.1 Request and upload limits

Control	Value (default)	Where
JSON request body	2 MB	JSON_BODY_LIMIT, <code>express.json</code>
Uploaded document or photo	10 MB per file, 10 files per request	UPLOAD_MAX_MB, UPLOAD_MAX_FILES; held in memory while checked (<code>lib/uploadLimits.js</code>)
Spreadsheet / CSV import, statement file	10 MB, one file	IMPORT_MAX_MB
Workbook decompression	50 MB per workbook part	IMPORT_MAX_INFLATED_MB (protection against compressed "zip bombs")
Import rows	20,000 rows per file; bulk upload screens <code>limits.bulk_upload_max_rows = 1,000</code>	IMPORT_MAX_ROWS, settings
Upload settings	images 2 MB (<code>uploads.image_max_bytes</code>), bulk files 10 MB (<code>uploads.bulk_max_bytes</code>)	settings group uploads
Web container request body	25 MB	<code>nginx.conf client_max_body_size</code>
List page size	default 10 (20 for notifications), maximum 500 per page	<code>lib/respond.js paging()</code>
Report rows	50,000 per generated file; default on-screen page 50; default date range 365 days	<code>reports.max_rows</code> , <code>reports.default_page_size</code> , <code>reports.default_range_days</code>
Global API rate limit	600 requests per 60 s per signed-in user (or per IP) per API process	<code>security.api_rate_limit</code>
Sign-in rate limit	10 attempts per 300 s per IP and per username per API process	<code>security.login_rate_limit</code>
Database connections	10 per API process	<code>db/pool.js</code> (max: 10, not configurable by environment)
E-mail throughput	50 messages per outbox run, every 5 minutes (600 per hour)	<code>lib/mailer.js</code> , job email-outbox

2.2 Performance mechanisms

- **Server-side paging** on lists (`LIMIT / OFFSET` with a total count) and on report runs; screens request one page at a time.
- **Indexes** on business numbers, status and date columns of the main tables, partial indexes for hot subsets (open direct-bill items, live refresh tokens, active entry matches, direct-billed policies) and a GIN index on generic master data (document 03).
- **Settings cache** in memory per process (`lib/settings.js`), so configuration is read from the database once per key.

- **Asynchronous work** outside the request: e-mail through the outbox, renewal notices through `job_queue`, scheduled reports and ageing through the scheduler.
- **Static assets** served by CloudFront / nginx with one-year caching of hashed files (`/static/`), `index.html` not cached.
- **Stateless API**: capacity scales horizontally by adding API instances behind the load balancer (document 07).

2.3 Known performance risks in the code

Risk	Detail and recommendation
CPU-bound document generation	PDF, XLSX and CSV files (up to 50,000 report rows) and spreadsheet imports are generated synchronously in the Node.js event loop of the API process. While a large report is written, other requests on the same instance wait. Recommended: keep <code>reports.max_rows</code> as is, run large scheduled reports on an instance with <code>SCHEDULER_ENABLED=true</code> that is not in the web target group, or move report generation to a worker (later change).
Uploads in memory	Up to 10 files of 10 MB per request are held in memory. Size the container memory for concurrent uploads (at least 1 GB of head-room per instance; 2 GB recommended).
Foreign keys without indexes	128 foreign keys have no supporting index (document 03). Joins and parent deletes on growing tables will degrade with volume.
Unbounded operational tables	<code>job_runs</code> (about 1,735 rows a day), <code>audit_log</code> , <code>login_history</code> , <code>notifications</code> , <code>email_outbox</code> , <code>refresh_tokens</code> have no purge (document 10); <code>job_runs</code> and <code>email_outbox</code> are scanned by the scheduler without a suitable index.
OFFSET paging	Deep pages of very large lists cost more with <code>OFFSET</code> ; acceptable for the page sizes used, review if lists exceed about 100,000 rows.
Per-process pool	With N instances the database sees up to 10 x N connections (plus scheduler lock connections); keep within the RDS <code>max_connections</code> of the chosen class.

3 Indicative load test

3.1 Method

The harness `docs/architecture/tools/loadtest.mjs` starts an in-process instance of the API (the same `createApp()` as production) on an ephemeral port of the development container, so the running system on ports 8000 and 5080 was not affected; it signs an access token for the administrator the way the backend tests do and drives read-only requests at a fixed concurrency for 15 seconds per endpoint, once with 10 and once with 50 concurrent clients. The per-process rate limiter was reset every 100 ms by the test hook `resetRateLimits()` so it did not cap the measurement.

Condition	Value
Date	29 September 2026
Host	Development container: 4 vCPU, 16 GB RAM, Linux; PostgreSQL 16.13 on the same host
API	Node.js v22.22.2, one process (one event loop), pool of 10 connections, <code>LOG_LEVEL=warn</code>
Data	Local test database at the time of the test (31 policies, 18 leads, 17 claims, 568 audit rows): test data , far smaller than production
Client	Same host (no network latency, no TLS, no load balancer)

Note: The results are **indicative only**: single small container, client and database on the same host, small test data set, no TLS and no network. They show the relative cost of the endpoints and that a single API process is CPU-bound on the event loop, not the capacity of the production environment.

3.2 Results

Clients: concurrent clients; Req/s: completed requests per second; latencies in milliseconds measured at the client; Non-2xx: responses other than 2xx.

Endpoint	Clients	Requests	Req/s	p50 ms	p95 ms	p99 ms	Max ms	Non-2xx
GET /health (readiness, DB ping)	10	27,833	1,855	4.5	10.2	17.8	100.1	6
GET /health/live (liveness)	10	56,719	3,781	2.2	5.1	8	29.9	0
GET /policies?page=1&perPage=10	10	5,939	396	23.2	41.4	59.6	127	0
GET /leads?page=1&perPage=10	10	11,807	787	11.5	20.7	33.4	82.4	0
GET /claims?page=1&perPage=10	10	7,150	476	19.6	31.4	52.6	87.4	0
GET /dashboard/executive	10	5,900	393	21.9	44.5	73.7	245.8	0
GET /notifications/unread-count	10	17,402	1,160	8.1	12.6	21	49.4	0
GET /search?q=santos	10	10,319	688	13	25.1	38.7	84.9	0
POST /reports/production-register/run (on-screen preview)	10	5,870	391	22.6	45.4	66.1	191.8	0
GET /health (readiness, DB ping)	50	34,913	2,326	18.1	39	51.7	545.1	3
GET /health/live (liveness)	50	34,246	2,282	20.2	35	46.3	283.5	0
GET /policies?page=1&perPage=10	50	6,392	423	107.8	167.8	286	484.6	0
GET /leads?page=1&perPage=10	50	11,848	788	58.6	91.3	122.3	174	0
GET /claims?page=1&perPage=10	50	8,083	537	85.7	132.1	167.1	240.8	0
GET /dashboard/executive	50	5,295	351	131.6	218	298.8	374.9	0

Endpoint	Clients	Requests	Req/s	p50 ms	p95 ms	p99 ms	Max ms	Non-2xx
GET /notifications/unread-count	50	19,720	1,314	34.3	61.4	98.1	149.3	0
GET /search?q=santos	50	8,568	569	83.1	134.3	177.5	255.6	0
POST /reports/production-register/run (on-screen preview)	50	5,748	382	122.1	199.1	236.2	305.1	0

3.3 Interpretation

- At 10 concurrent clients a single API process served about 390 to 790 requests per second on list, dashboard, search and report-preview endpoints with 95th-percentile latencies between 21 and 45 ms, and about 1,160 per second on the notification unread count polled by every signed-in browser.
- At 50 concurrent clients throughput stayed about the same while latency grew about five times (95th percentile 91 to 218 ms): one Node.js process is saturated at a few hundred requests per second of this mix, so capacity grows by adding processes (instances), not by concurrency.
- The executive dashboard, policy list and report preview are the most expensive of the measured reads (about 2.5 ms of CPU per request each on this host); the unread-count poll is cheap but frequent (one request per signed-in user every 30 seconds).
- 9 of about 62,700 readiness checks ([GET /api/health](#)) answered 503 during the runs (none on the liveness check); the cause was not isolated in this test. A load balancer marks a target unhealthy only after several consecutive failures, so isolated 503s do not remove an instance, but this should be re-checked in the UAT test with production-like health check settings.
- With production data volumes (hundreds of thousands of rows) the database share of each request will grow; the UAT test must use a volume-loaded database.

4 Data growth and storage

4.1 Current relative weight (test data)

Table	Rows (test data)	Total size KB	Average row bytes	Retention class
job_runs	961	248	145	Log / audit
audit_log	574	400	424	Log / audit
refresh_tokens	352	248	153	Temporary
login_history	351	168	169	Log / audit
master_records	317	384	385	Reference
journal_lines	286	152	180	Financial
app_settings	273	128	190	Reference
role_permissions	226	56	32	Reference
gl_accounts	128	128	166	Reference
journal_vouchers	117	192	338	Financial
generated_reports	115	144	525	Temporary
claim_field_changes	63	80	132	Log / audit
claim_history	54	48	90	Log / audit
commissions	53	120	332	Financial
master_types	52	96	900	Reference

4.2 Growth estimate

The estimate multiplies the measured average row sizes of the test database by assumed business volumes. Index and page overhead roughly doubles the heap size.

Assumption (Recommended / to be confirmed by the business)	Value	Basis
Named users / concurrent users at peak	200 / 80	to be confirmed
Policies issued per year (new + renewal)	50,000	to be confirmed
Endorsements, claims per year	10,000 / 3,000	to be confirmed
Uploaded documents and photos per policy	4 files, 0.5 MB average	to be confirmed
Generated files (reports, statements, schedules)	2 GB per year, 90-day retention for reports	settings <code>reports.retention_days</code>

Item	Estimate per year	Calculation
Transaction rows per policy (quotation 1.4 KB, policy 0.8 KB, bill, booking and receipt journals with lines, commission, receipt and application, collection item, remittance line)	about 5 KB heap	measured average row sizes
Audit, notification, e-mail and document rows per policy (about 10 audit rows at 0.4 KB, 2 notifications, 2 e-mails, 4 document rows)	about 6 KB heap	measured average row sizes
Database growth from policies, incl. index overhead	about 1.1 GB	50,000 x 11 KB x 2

Item	Estimate per year	Calculation
Endorsements, claims, renewals, reinsurance, accounting of other flows	about 0.4 GB	estimate
Scheduler history (<code>job_runs</code> , 1,735 rows a day at 145 bytes, x 2)	about 0.2 GB	from the job schedule
Sign-in history, refresh tokens (200 users, 4 sessions a day)	under 0.1 GB	measured row sizes
Database total	about 2 GB per year (plus WAL and backups)	
File store (uploads)	about 100 GB per year	50,000 x 4 x 0.5 MB, rounded up

The database stays small; the file store dominates storage growth and backup volume.

5 Sizing recommendation

Recommended / to be confirmed by the business and DevOps: Starting sizes for production in AWS ap-southeast-1, based on the assumptions above; confirm with the UAT performance test and adjust after the first months of operation.

Component	Recommended start	Scaling rule
API (ECS Fargate tasks)	2 tasks of 1 vCPU and 2 GB, in two AZs; one task with <code>SCHEDULER_ENABLED=true</code> is enough, both may run it (advisory lock)	Target tracking on average CPU 60 %; maximum 6 tasks; add a task per ~250 requests per second of peak load
Load balancer	ALB, idle timeout 120 s (large report downloads and imports), health check <code>/api/health</code> every 15 s, 3 failures to mark unhealthy	Managed
Database	Amazon RDS for PostgreSQL 16, <code>db.m7g.large</code> (2 vCPU, 8 GB) Multi-AZ; a <code>db.t4g.medium</code> Multi-AZ is sufficient for UAT	Scale the class vertically on sustained CPU above 60 % or when connections exceed 70 % of <code>max_connections</code>
Database storage	gp3, 100 GB with storage autoscaling to 500 GB	Alarm at 80 % used
File store	Amazon EFS, General Purpose, Elastic throughput, lifecycle to Infrequent Access after 90 days	Grows automatically; alarm on monthly growth
Static front end	S3 + CloudFront	Managed
Connections	10 per API task (+1 per running job)	6 tasks = about 70 connections, well within the RDS limit

6 Performance test plan for UAT

6.1 Objectives

1. Confirm that the production sizing meets the response-time targets below at the expected peak load plus 50 % head-room.
2. Find the saturation point of one API task and the scaling behaviour with two and four tasks.
3. Confirm that scheduled jobs, report generation and bulk imports do not degrade interactive response times beyond the targets.
4. Establish a performance baseline for regression tests of later releases.

6.2 Response-time targets

Transaction class (Recommended / to be confirmed by the business)	95th percentile	Maximum error rate
Navigation, lists, details, search, notification count	1.0 s	0.1 %
Save / submit (lead, quotation, receipt, voucher, claim)	2.0 s	0.1 %
Quotation pricing, policy issue (incl. billing and journals)	3.0 s	0.1 %
Dashboards, on-screen reports (one page)	3.0 s	0.5 %
Report file generation up to 50,000 rows, bulk import 1,000 rows	60 s	1 %

6.3 Approach

Step	Content
Environment	UAT on the production sizing (chapter 5), with RDS, EFS, ALB and CloudFront as in production; <code>LOG_LEVEL=info</code>
Data	Volume-loaded copy: at least one year of forecast volume (for example 50,000 policies with bills, receipts, journals, commissions; 3,000 claims; 200 users); generated with a data script on UAT (never production data without masking)
Tool	k6, JMeter or Artillery scripts per persona, using the API (tokens obtained through <code>POST /api/auth/login</code> for test users; one user per virtual user to avoid the per-user rate limit, or <code>security.api_rate_limit</code> raised for the test)
Workload mix	Agent (search, quotation, policy view), finance (receipts, journal vouchers, remittance), claims, dashboards and reports, the 30-second notification poll per user
Test types	Baseline (1 user), load (expected peak 80 concurrent users for 1 hour), stress (increase to failure), soak (peak load for 8 hours with the scheduler running), spike (5 x peak for 5 minutes)
Measurements	Client latencies and errors; ALB target response time and 5xx; ECS CPU and memory; RDS CPU, connections, IOPS, top SQL (Performance Insights); <code>job_runs</code> durations; event-loop delay
Exit criteria	Targets met at peak + 50 %; no memory growth in the soak test; no errors other than intended 429s; database CPU below 60 % at peak
Deliverable	Test report with the measured capacity per task, the confirmed sizing and a list of slow SQL statements with index or query fixes

7 Related documents

This document is one of eleven that together form the BrokerVerse Solution Architecture. They are maintained and issued together (source and build: [docs/architecture/README.md](#)).

No.	Document	File
01	Solution Component Diagram and Application Architecture	01_BrokerVerse_Solution_Component_Diagram_Application_Architecture.pdf
02	Database Design	02_BrokerVerse_Database_Design.pdf
03	Database Inventory	03_BrokerVerse_Database_Inventory.pdf
04	Technology Stack	04_BrokerVerse_Technology_Stack.pdf
05	Shared Service Components	05_BrokerVerse_Shared_Service_Components.pdf
06	Capacity and Performance (this document)	06_BrokerVerse_Capacity_and_Performance.pdf
07	High Availability and Resiliency	07_BrokerVerse_High_Availability_and_Resiliency.pdf
08	RTO and RPO	08_BrokerVerse_RTO_and_RPO.pdf
09	Backup and Recovery	09_BrokerVerse_Backup_and_Recovery.pdf
10	Data Archival, Housekeeping and Restoration	10_BrokerVerse_Data_Archival_Housekeeping_and_Restoration.pdf
11	Monitoring	11_BrokerVerse_Monitoring.pdf

8 Glossary

Terms and abbreviations used in this document.

Term	Meaning
Advisory lock	PostgreSQL application-level lock (<code>pg_try_advisory_lock</code>) used by the scheduler so a job runs on one API instance at a time.
ALB	Application Load Balancer (AWS): distributes HTTPS requests over the API containers and checks their health.
API	Application programming interface; here the BrokerVerse REST API under <code>/api</code> served by the Node.js backend.
AZ	Availability Zone: an isolated data-centre group inside an AWS region.
CloudFront	AWS content delivery network that serves the single-page application and can route <code>/api</code> to the backend.
ECS / Fargate	Elastic Container Service (AWS) and its serverless compute mode for running the API containers.
EFS	Elastic File System (AWS): shared, multi-AZ network file system proposed for the upload directory.
GIN	Generalised inverted index (PostgreSQL), used for JSONB searches.
Multi-AZ	Deployment with a synchronous standby in a second Availability Zone and automatic failover (RDS).
nginx	Web server used in the web container to serve the SPA and forward <code>/api</code> to the API container.
p50 / p95 / p99	Latency percentiles: the time within which 50 %, 95 % or 99 % of requests completed.
RDS	Amazon Relational Database Service, the managed PostgreSQL target.
Receivable / bill	Premium bill (INV-) raised when a policy, endorsement or renewal is issued.
Remittance	Payment of collected premium, net of commission, from the broker to the insurer.
RPO	Recovery point objective: the maximum acceptable data loss, measured in time.
RTO	Recovery time objective: the maximum acceptable time to restore service.
S3	Amazon Simple Storage Service: object storage hosting the SPA build (and proposed archive store).
TLS	Transport Layer Security (HTTPS).
UAT	User acceptance testing environment.
WAL	Write-ahead log: PostgreSQL change log used for replication and point-in-time recovery.